Yes. **If you use Azure Cosmos DB with the Gremlin API, the graph retrieval is performed using Gremlin queries.**

Think of it like this:

Graph Storage	Query Language
NetworkX	Python (BFS, DFS, PageRank, etc.)
Neo4j	Cypher
Azure Cosmos DB (Gremlin API)	**Gremlin**
RDF Triple Store	SPARQL

Azure Cosmos DB (Brief Overview)

What is Azure Cosmos DB?

Azure Cosmos DB is Microsoft's **fully managed, globally distributed NoSQL database service**. It is designed to store and retrieve very large amounts of data with low latency, high availability, and automatic scaling.

Unlike traditional relational databases (MySQL, PostgreSQL), Cosmos DB supports multiple data models, including **document**, **key-value**, **column-family**, and **graph**.

For a **Graph RAG** application, Cosmos DB uses the **Gremlin API**, which stores data as a graph of **vertices (nodes)** and **edges (relationships)**.

Why use Cosmos DB?

Traditional databases are good at storing rows and tables.

Example:

Employee Table

ID Name

1 John

Department Table

ID Name

10 AI

To find relationships, SQL uses JOIN operations.

Graph databases are different.

They directly store relationships.

John

  |

works_in

  |

AI Department

This makes relationship traversal much faster.

Data Model

Cosmos DB (Gremlin API) stores data as:

Vertex (Node)

Represents an entity.

Example:

Engine

Fuel Pump

Injector

Hydraulic System

Edge

Represents the relationship.

Engine

  |

 uses

|

Fuel Pump

Another example

Fuel Pump

|

supplies

|

Injector

Graph Representation

uses

Engine ----------> Fuel Pump

|

|

supplies

|

▼

Injector

APIs Supported by Cosmos DB

Cosmos DB supports multiple APIs.

API	**Purpose**
SQL API	JSON Documents
MongoDB API	MongoDB Compatible
Cassandra API	Column Database
Table API	Key-Value Storage
Gremlin API	Graph Database

For Graph RAG, we use **Gremlin API**.

Query Language

The Gremlin API uses **Gremlin Query Language**.

Example:

Find the Engine node.

g.V().has('title','Engine')

Find all connected nodes.

g.V().

has('title','Engine').

out()

Two-hop traversal.

g.V().

has('title','Engine').

repeat(out()).

times(2)

Cosmos DB in Your Graph RAG

Your XML Graph RAG pipeline would look like this:

XML Files

|

▼

Extract Content

|

▼

Extract Hyperlinks

|

▼

LLM extracts predicates

|

▼

Create RDF Triples

|

▼

Store in Cosmos DB

|

▼

Gremlin Query

|

▼

Retrieve Related Documents

|

▼

Azure OpenAI

|

▼

Generate Answer

Retrieval Process

When a user asks a question:

How does the fuel pump work?

The system performs:

User Question

|

▼

Identify starting node

|

▼

Execute Gremlin Query

|

▼

Retrieve connected nodes

|

▼

Fetch XML contents

|

▼

Pass context to LLM

|

▼

Generate answer

Advantages

- Fully managed Azure service
- Automatic scaling
- Persistent graph storage
- Handles millions of nodes and relationships
- Fast graph traversal using Gremlin
- High availability and global replication
- Integrates well with Azure OpenAI and Azure services
- Suitable for enterprise Graph RAG applications

Limitations

- Paid cloud service

- Requires Azure account and configuration
- Gremlin query language has a learning curve
- Visualization is not built in (external tools are typically used)
- More complex than using an in-memory library like NetworkX

Cosmos DB vs NetworkX

Feature	NetworkX	Cosmos DB
Type	Python Graph Library	Managed Graph Database
Storage	In Memory	Persistent Cloud Storage
Query	Python (BFS, DFS, etc.)	Gremlin
Scalability	Limited by RAM	Millions to Billions of Nodes
Multi-user Support	No	Yes
Cost	Free	Paid
Best For	Research, POC, Algorithms	Production, Enterprise Systems

---

Which should you use?

For your project, a practical approach is:

Phase 1 (Current POC)

- XML Parsing
- RDF Triple Generation
- NetworkX Knowledge Graph
- Graph Retrieval (BFS/DFS/PageRank)
- LLM Answer Generation

This is easy to build, debug, and evaluate.

Phase 2 (Production)

- XML Parsing
- RDF Triple Generation

- Azure Cosmos DB (Gremlin API)
- Gremlin Graph Traversal
- Hybrid Retrieval (Graph + Vector Search)
- LLM Answer Generation

This provides persistence, scalability, and enterprise-grade performance.

Summary

Azure Cosmos DB is a **cloud-native graph database** that stores knowledge as **nodes and relationships** and uses the **Gremlin query language** for graph traversal. In a Graph RAG system, it replaces an in-memory graph (such as NetworkX) with a scalable, persistent graph database. For your current proof of concept, NetworkX is the simpler choice. If your solution later needs to support large-scale data, multiple users, or production deployment, Cosmos DB is a strong option.